

Atividade 1.01 – Criando um App para Criar Cores RGB

Objetivo da Atividade

Nesta atividade, vamos explorar um cenário que utiliza validação. Suponha que você tenha sido encarregado de criar um app que demonstra como os canais RGB de vermelho, verde e azul são combinados no espaço de cores RGB para criar uma cor.

Cada canal RGB deve ser adicionado como dois caracteres hexadecimais, onde cada caractere pode ter um valor de 0–9, A–F ou a–f. Os valores serão então combinados para produzir uma string hexadecimal de seis caracteres que é exibida como uma cor dentro do app.

Descrição da Atividade

O objetivo desta atividade é produzir um formulário com campos editáveis nos quais o usuário pode adicionar dois valores hexadecimais para cada cor. Após preencher todos os três campos, o usuário deve clicar em um botão que pega os três valores e os concatena para criar uma string de cor hexadecimal. Isso deve ser convertido para uma cor e exibido na interface do app.

Passos para Completar a Atividade

1. Criar um Novo Projeto no Android Studio

Crie um novo projeto no Android Studio, assim como você fez no Exercício 1.01 – Criando um projeto Android Studio para seu app.

2. Adicionar um Composable de Título

Adicione um composable `Text` que funcione como título da sua aplicação.

3. Adicionar uma Descrição Breve

Adicione uma breve descrição informando ao usuário como completar o formulário. A descrição deve explicar que o usuário precisa inserir dois caracteres hexadecimais (0–9, A–F ou a–f) para cada canal de cor.

4. Adicionar Propriedades MutableState

Adicione três propriedades `MutableState` para cada uma das três cores e outra para uma cor padrão usando `mutableStateOf(androidx.compose.ui.graphics.Color.White)` .

Exemplo:

Kotlin

```
val redChannel = remember { mutableStateOf("") }
val greenChannel = remember { mutableStateOf("") }
val blueChannel = remember { mutableStateOf("") }
val displayColor = remember { mutableStateOf(Color.White) }
```

5. Adicionar Campos de Entrada (OutlinedTextField)

Adicione três composables `OutlinedTextField` do Material com rótulos de "Canal Vermelho", "Canal Verde" e "Canal Azul", inicializados com uma string vazia.

6. Adicionar Restrição de Entrada

Adicione uma restrição a cada campo para permitir a entrada de apenas dois caracteres hexadecimais. Você pode conseguir isso com a seguinte função:

Kotlin

```
fun isValidHexInput(input: String): Boolean {
    return input.filter {
        it in '0'..'9' ||
        it in 'A'..'F' ||
        it in 'a'..'f'
    }.length == 2
}
```

Como usar: Valide o input do usuário sempre que o campo for alterado e atualize o estado apenas se a entrada for válida.

7. Adicionar um Botão de Ação

Adicione um botão que pega os inputs dos três campos de cor. Quando clicado, o botão deve concatenar os três valores para criar uma string de cor hexadecimal.

8. Adicionar uma Visualização para Exibir a Cor

Adicione uma visualização que exibe a cor produzida no layout. Isso pode ser conseguido criando uma string de cor começando com um caractere `#` e usando templates de string Kotlin para concatenar as cores juntas.

Converta a string para uma cor usando `Color(colorString.toColorInt())` e defina-a como o fundo de um `Text` com `Modifier.background(colorToDisplay).padding(24.dp)`.

9. Exibir a Cor RGB Criada

Por fim, exiba a cor RGB criada a partir dos três canais no layout. O resultado final deve ser semelhante ao da imagem de referência (a cor variará dependendo dos inputs).

Importante: A aplicação não precisa ser idêntica à imagem fornecida – **seus alunos devem ser criativos** e adicionar suas próprias melhorias visuais e funcionais!

Referência Visual

A imagem de referência mostra um exemplo de como o app pode parecer:

- Um campo de entrada para o Canal Vermelho (com valor "FF" como exemplo)
- Um campo de entrada para o Canal Verde (com valor "44" como exemplo)
- Um campo de entrada para o Canal Azul (com valor "00" como exemplo)
- Um botão para criar a cor RGB
- Um painel que exibe a cor criada

Dicas para Melhorias Criativas

Seus alunos podem considerar as seguintes melhorias:

Validação e Feedback:

- Mostrar mensagens de erro quando a entrada for inválida
- Exibir a cor em tempo real conforme o usuário digita (se os valores forem válidos)
- Indicar visualmente quando um campo está completo

Interface do Usuário:

- Usar cores diferentes para os rótulos dos campos (vermelho, verde, azul)
- Adicionar ícones ou emojis para representar cada canal
- Criar um design mais atraente com gradientes ou sombras

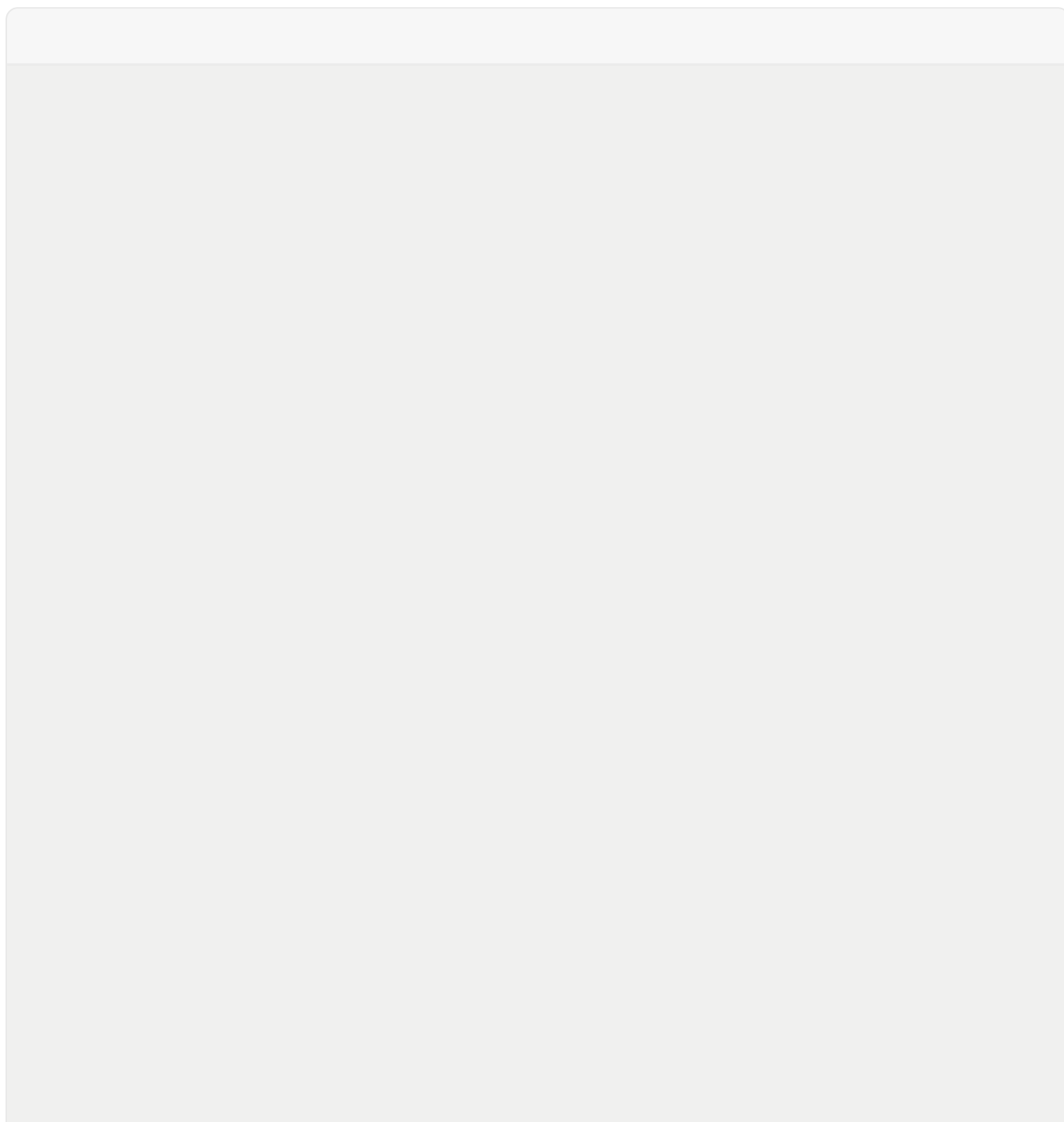
Funcionalidades Adicionais:

- Adicionar um botão para copiar o código hexadecimal para a área de transferência
- Permitir que o usuário salve cores favoritas
- Exibir o código hexadecimal completo (#RRGGBB) em um campo de texto
- Adicionar um seletor de cores como alternativa

- Implementar um histórico das cores criadas

Acessibilidade:

- Adicionar descrições de acessibilidade (contentDescription)
- Usar contraste adequado de cores
- Permitir entrada via teclado numérico



Critérios de Avaliação Sugeridos

- **Funcionalidade:** O app cria e exibe cores RGB corretamente
- **Validação:** A entrada é validada corretamente para caracteres hexadecimais
- **Interface:** A interface é clara e fácil de usar
- **Criatividade:** O aluno adiciona melhorias ou recursos próprios
- **Código:** O código é bem estruturado e comentado

Conclusão

Esta atividade permite que os alunos pratiquem conceitos importantes de desenvolvimento Android, incluindo:

- Uso de `MutableState` para gerenciar estado
- Validação de entrada do usuário
- Composables do Material Design
- Manipulação de cores em Kotlin
- Criatividade no design de interface

Encorajamos seus alunos a experimentarem e criarem suas próprias versões únicas desta aplicação!

Traduzido e adaptado para português. Última atualização: 17 de fevereiro de 2026